

**МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**

Отчёт по проекту GoToPython

Выполнили:

студент 5 курса 9 группы Чаленко М.М,

студент 5 курса 9 группы Андреев Н.Н.

студент 5 курса 9 группы Щербаков К.Н.

Воронеж 2025

1. Общее описание

GoToPy - это интерпретатор подмножества языка Go, реализованный на Python. Проект демонстрирует основные принципы создания интерпретатора языка программирования, включая лексический анализ, синтаксический анализ и выполнение кода.

2. Подмножества языка Go:

Требуемая функциональность языка представлена в таблице ниже:

Функциональность	Поддерживается обязательно	Поддерживается по желанию/возможности
Типы данных	int, bool, byte, одномерные массивы	string, float64, rune, []T, map, struct
Операции	+, -, *, /, %, ==, !=, <, >, <=, >=, !, логические &&, `	
Операторы управления	if { } else { }, for, switch, return	break, continue, goto, fallthrough
Функции	Объявление функций, передача простых типов по значению	Замыкания, передача по ссылке (*T), возвращение нескольких значений
Ввод/Вывод	fmt.Println, fmt.Scan	bufio, os, fmt.Sprintf, форматированный ввод/вывод
Системные функции	len, cap, append для массивов и срезов	copy, make, new, delete и т.п.

3. Детальное описание компонентов

3.1 Лексический анализатор (go_lexer.py)

3.1.1 Токены

NUMBER, IDENT, STRING, BOOL, PLUS, MINUS, TIMES, DIVIDE, MOD, EQ, NE, LT, LE, GT, GE, ASSIGN, DECLARE, AND, OR, NOT, LPAREN, RPAREN, LBRACE, RBRACE, SEMI, COMMA, COLON, VAR, PRINT, IF, ELSE, FOR, WHILE, TYPE, FUNC, RETURN

3.1.2 Зарезервированные слова

```
{'var': 'VAR', 'true': 'BOOL', 'false': 'BOOL', 'print': 'PRINT', 'if': 'IF', 'else': 'ELSE', 'for': 'FOR', 'while': 'WHILE', 'int': 'TYPE', 'float': 'TYPE', 'bool': 'TYPE', 'string': 'TYPE', 'func': 'FUNC', 'return': 'RETURN'}
```

3.2 Синтаксический анализатор (go_parser.py)

3.2.1 Грамматика языка

Программа: последовательность объявлений и операторов

Объявления: переменные и функции

Операторы: присваивание, условные, циклы, вызовы функций

Выражения: арифметические, логические, сравнения

3.2.2 Правила разбора

Обработка приоритетов операторов, проверка синтаксической корректности и построение AST.

3.3 Абстрактное синтаксическое дерево (go_ast.py)

3.3.1 Основные узлы

- ProgramNode
- BlockNode
- VarDeclNode
- AssignNode
- IfNode
- ForNode
- WhileNode
- PrintNode

- BinOpNode
- UnOpNode
- FuncDeclNode
- FuncCallNode
- ReturnNode

3.4 Интерпретатор (go_interpreter.py)

3.4.1 Управление памятью

Иерархическая структура областей видимости, поддержка вложенных функций и управление переменными.

3.4.2 Выполнение кода

1. Последовательное выполнение операторов
2. Обработка условных переходов
3. Выполнение циклов
4. Вызов функций
5. Обработка возвращаемых значений

3.4.3 Обработка ошибок

Проверка типов, проверка существования переменных и корректности операций.

4. Примеры использования

Простой пример

```
var x: int = 10;  
var y: int = 5;  
print(x + y);
```

Функция с условиями

```
func isPrime(n: int): bool {  
    if (n <= 1) {  
        return false;  
    }  
    var i: int = 2;
```

```
while (i * i <= n) {  
    if (n % i == 0) {  
        return false;  
    }  
    i = i + 1;  
}  
return true;  
}
```

Работа со строками

```
var name: string = "Иван";  
var age: int = 25;  
var greeting: string = "Привет, " + name + "!";  
print(greeting);
```

5. Ограничения

- Нет поддержки массивов и структур
- Ограниченная поддержка типов данных
- Нет поддержки пакетов и импортов
- Нет обработки исключений
- Нет поддержки горутин и каналов

6. Тестирование

Проект включает набор тестов, проверяющих базовые арифметические операции, условные операторы, циклы, функции, работу со строками, булевы операции и комплексные сценарии использования.

7. Возможные улучшения

6. Добавление поддержки массивов и структур
7. Реализация системы типов
8. Добавление поддержки пакетов
9. Реализация обработки исключений
10. Оптимизация производительности
11. Добавление поддержки стандартной библиотеки

12. Улучшение системы обработки ошибок

13. Добавление поддержки отладки